群报数小程序——修改用户名用例描述

用例名称：

修改个人账号用户名

参与者：

群报数小程序用户（主要参与者），群报数系统服务器（次要参与者）

目标：

用户成功更新自己的显示昵称（用户名）。

范围：

群报数微信小程序

前置条件：

用户已通过微信授权成功登录小程序，并进入“我的/个人中心”页面。

后置条件：

用户的个人信息数据库被更新，修改记录写入系统日志。

主事件流

1. 用户在“我的”页面，点击“个人资料”选项。
2. 系统展示当前用户的基本信息（头像、当前昵称）。
3. 用户选择“修改昵称”。
4. (如果是修改用户名)：用户点击昵称，系统弹出输入框；用户输入新昵称并点击“完成”。
5. 系统对用户输入的内容进行合法性校验（如：昵称是否包含敏感词、长度是否符合要求）。
6. 系统向服务器发起数据更新请求。
7. 系统提示“昵称修改成功”，并自动刷新页面显示最新的用户名。

备选事件流（异常情况处理）

备选流 A

（输入格式错误）：步骤4中，若新用户名过长/含违规词，或新密码过于简单，系统拒绝提交，并在输入框下方红字提示“格式不符合要求，请重新输入”，返回步骤3。

备选流 B

（原密码错误）：在修改密码时，若用户输入的原密码与数据库不匹配，系统提示“原密码错误”，并限制连续错误次数，返回步骤3。

备选流 C

（网络异常）：步骤5中，若网络断开或服务器无响应，系统提示“网络连接超时，请稍后重试”，暂不保存修改。

图书馆管理系统

1、用例图 (Use Case Diagram)

```
left to right direction
skinparam packageStyle rectangle

actor "读者" as Reader
actor "图书管理员" as Librarian

rectangle "图书馆管理系统" {
  usecase "查询图书" as UC1
  usecase "借阅图书" as UC2
  usecase "归还图书" as UC3
  usecase "管理图书信息" as UC4
  usecase "处理逾期罚款" as UC5
}

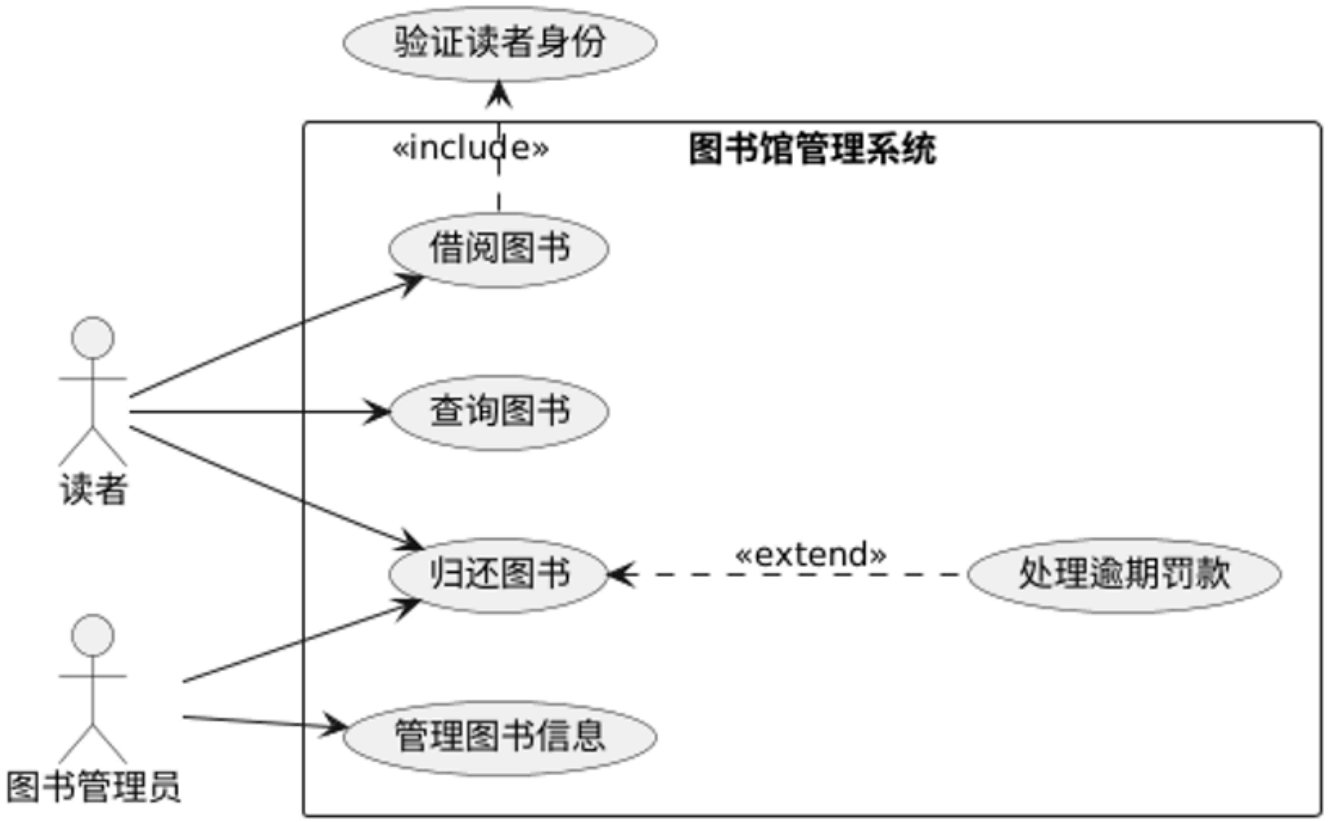
Reader --> UC1
Reader --> UC2
Reader --> UC3

Librarian --> UC4
Librarian --> UC3

' 表示包含关系（借书必须验证身份）
usecase "验证读者身份" as UC6
UC2 .> UC6 : <<include>>

' 表示扩展关系（还书时如果超期，则扩展出交罚款）
UC3 <.. UC5 : <<extend>>

@enduml
```

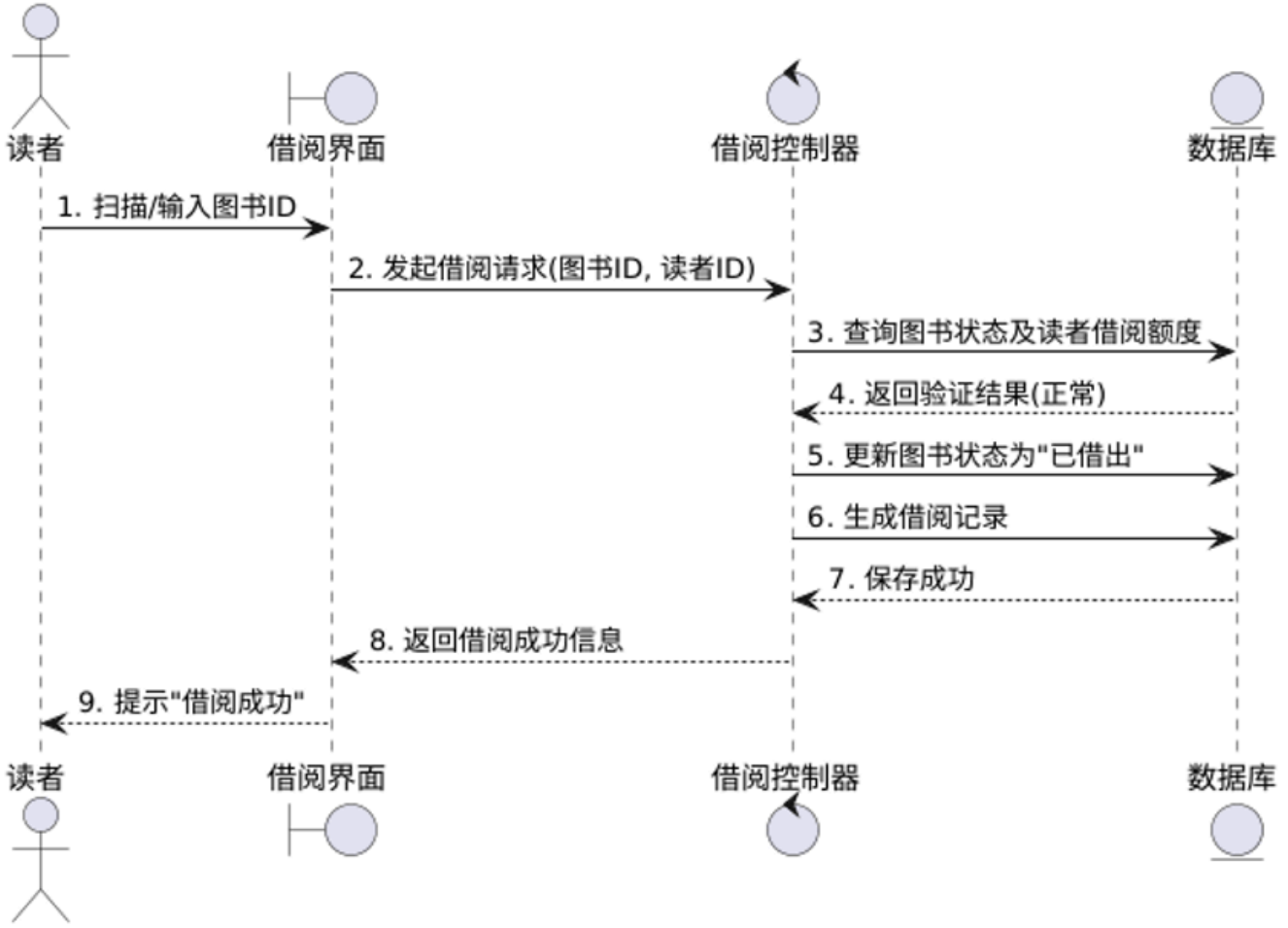


2、时序图 (Sequence Diagram)

```
actor 读者 as Reader
boundary "借阅界面" as UI
control "借阅控制器" as Controller
entity "数据库" as DB

Reader -> UI : 1. 扫描/输入图书ID
UI -> Controller : 2. 发起借阅请求(图书ID, 读者ID)
Controller -> DB : 3. 查询图书状态及读者借阅额度
DB --> Controller : 4. 返回验证结果(正常)
Controller -> DB : 5. 更新图书状态为"已借出"
Controller -> DB : 6. 生成借阅记录
DB --> Controller : 7. 保存成功
Controller --> UI : 8. 返回借阅成功信息
UI --> Reader : 9. 提示"借阅成功"

@enduml
```



3、类图 (Class Diagram)

```
class User {
  - userId: String
  - name: String
  - password: String
  + login()
  + logout()
}

class Reader {
  - readerId: String
  - maxBorrowLimit: int
  + borrowBook()
  + returnBook()
}

class Librarian {
  - adminId: String
  + manageBook()
  + handleReturn()
}

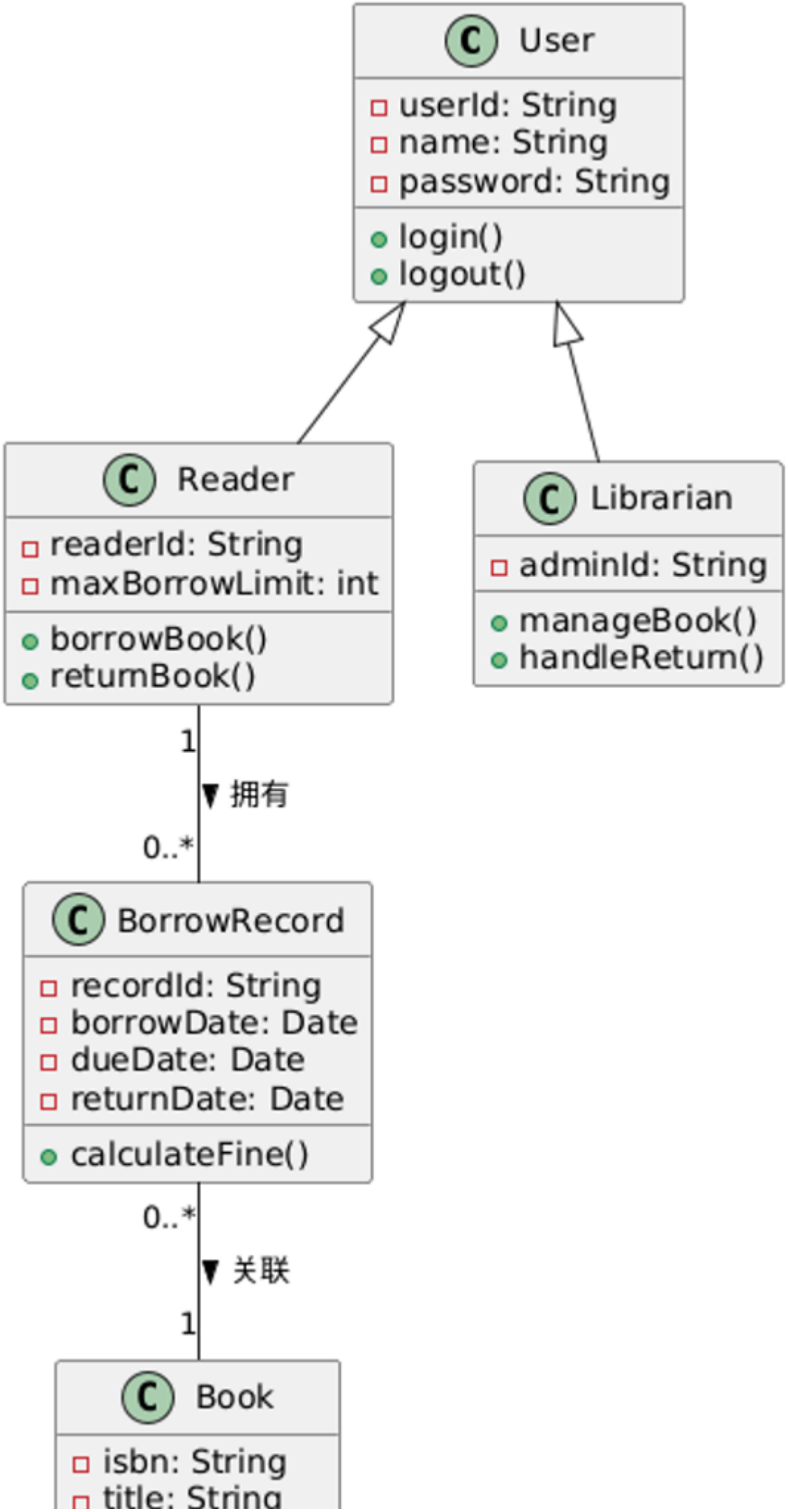
class Book {
  - isbn: String
```

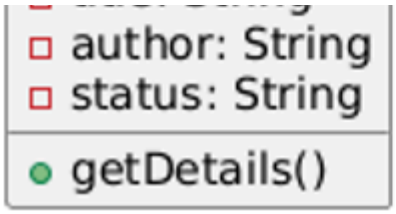
```
  - title: String
  - author: String
  - status: String
  + getDetails()
}

class BorrowRecord {
  - recordId: String
  - borrowDate: Date
  - dueDate: Date
  - returnDate: Date
  + calculateFine()
}

User <|-- Reader
User <|-- Librarian
Reader "1" -- "0..*" BorrowRecord : 拥有 >
BorrowRecord "0..*" -- "1" Book : 关联 >

@enduml
```


User
userId: String
name: String
password: String
login()
logout()
Reader
readerId: String
maxBorrowLimit: int
borrowBook()
returnBook()
Librarian
adminId: String
manageBook()
handleReturn()
1
拥有
0..*
BorrowRecord
recordId: String
borrowDate: Date
dueDate: Date
returnDate: Date
calculateFine()
0..*
关联
1
Book
isbn: String
title: String



4、对象图 (Object Diagram)

```
skinparam objectFontSize 14

' 定义具体的读者对象
object "张三: Reader" as r1 {
  readerId = "R1001"
  name = "张三"
  maxBorrowLimit = 5
}

' 定义具体的图书对象
object "《UML系统分析与设计》: Book" as b1 {
  isbn = "978-7-111-12345-6"
  title = "UML系统分析与设计"
  author = "Craig Larman"
  status = "已借出"
}

' 定义具体的借阅记录对象
object "记录001: BorrowRecord" as br1 {
  recordId = "BR20231024-001"
  borrowDate = "2023-10-24"
  dueDate = "2023-11-24"
  returnDate = null
}

' 对象之间的链接（对应类图中的关联关系）
r1 -- br1 : 拥有 >
br1 -- b1 : 关联 >

@enduml
```

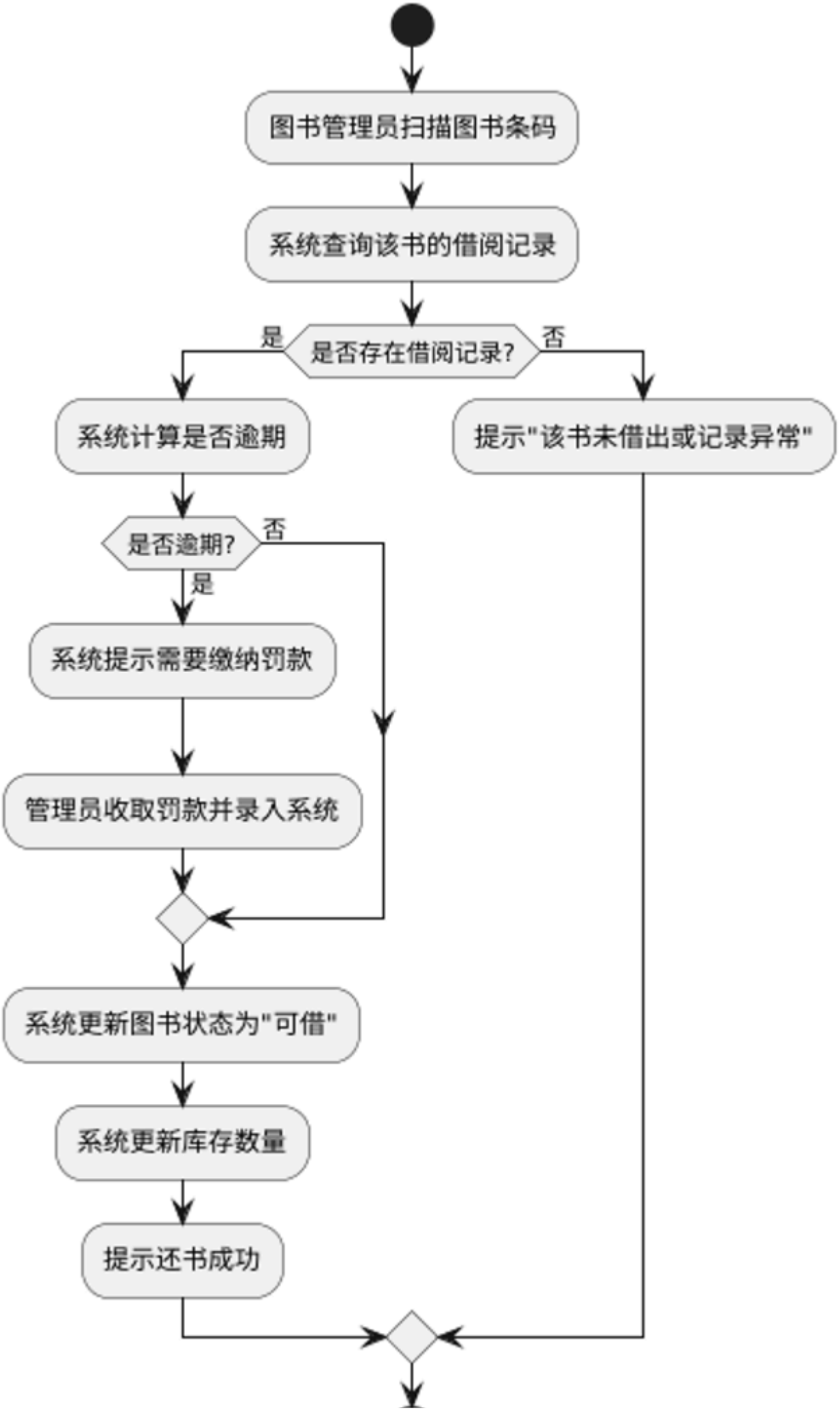


5、活动图 (Activity Diagram)

```
start
:图书管理员扫描图书条码;
:系统查询该书的借阅记录;
if (是否存在借阅记录?) then (是)
  :系统计算是否逾期;
  if (是否逾期?) then (是)
    :系统提示需要缴纳罚款;
    :管理员收取罚款并录入系统;
  else (否)
  endif
  :系统更新图书状态为"可借";
  :系统更新库存数量;
  :提示还书成功;
```

```
else (否)
  :提示"该书未借出或记录异常";
endif
stop

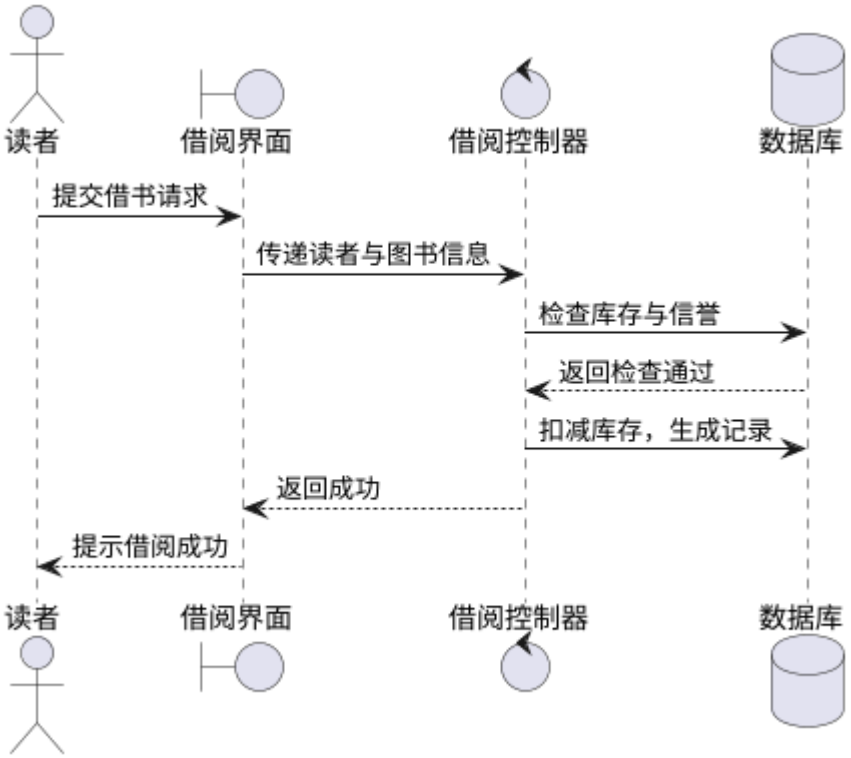
@enduml
```


图书管理员扫描图书条码
系统查询该书的借阅记录
是否存在借阅记录?
是
否
系统计算是否逾期
提示"该书未借出或记录异常"
是否逾期?
否
是
系统提示需要缴纳罚款
管理员收取罚款并录入系统
系统更新图书状态为"可借"
系统更新库存数量
提示还书成功

6.Legacy Syntax (旧版语法)

```
' Legacy Syntax Example (简化的旧版时序图写法)
actor 读者
boundary 借阅界面
control 借阅控制器
database 数据库

读者 -> 借阅界面 : 提交借书请求
借阅界面 -> 借阅控制器 : 传递读者与图书信息
借阅控制器 -> 数据库 : 检查库存与信誉
数据库 --> 借阅控制器 : 返回检查通过
借阅控制器 -> 数据库 : 扣减库存，生成记录
借阅控制器 --> 借阅界面 : 返回成功
借阅界面 --> 读者 : 提示借阅成功
@enduml
```



7、Component Diagram (组件图)

```
package "图书馆管理系统" {
  [读者客户端 (Web/小程序)] as ReaderApp
  [管理员后台] as AdminApp

  [借阅管理组件] as BorrowComp
```

```
  [图书管理组件] as BookComp
  [用户管理组件] as UserComp
  [支付/罚款组件] as PayComp

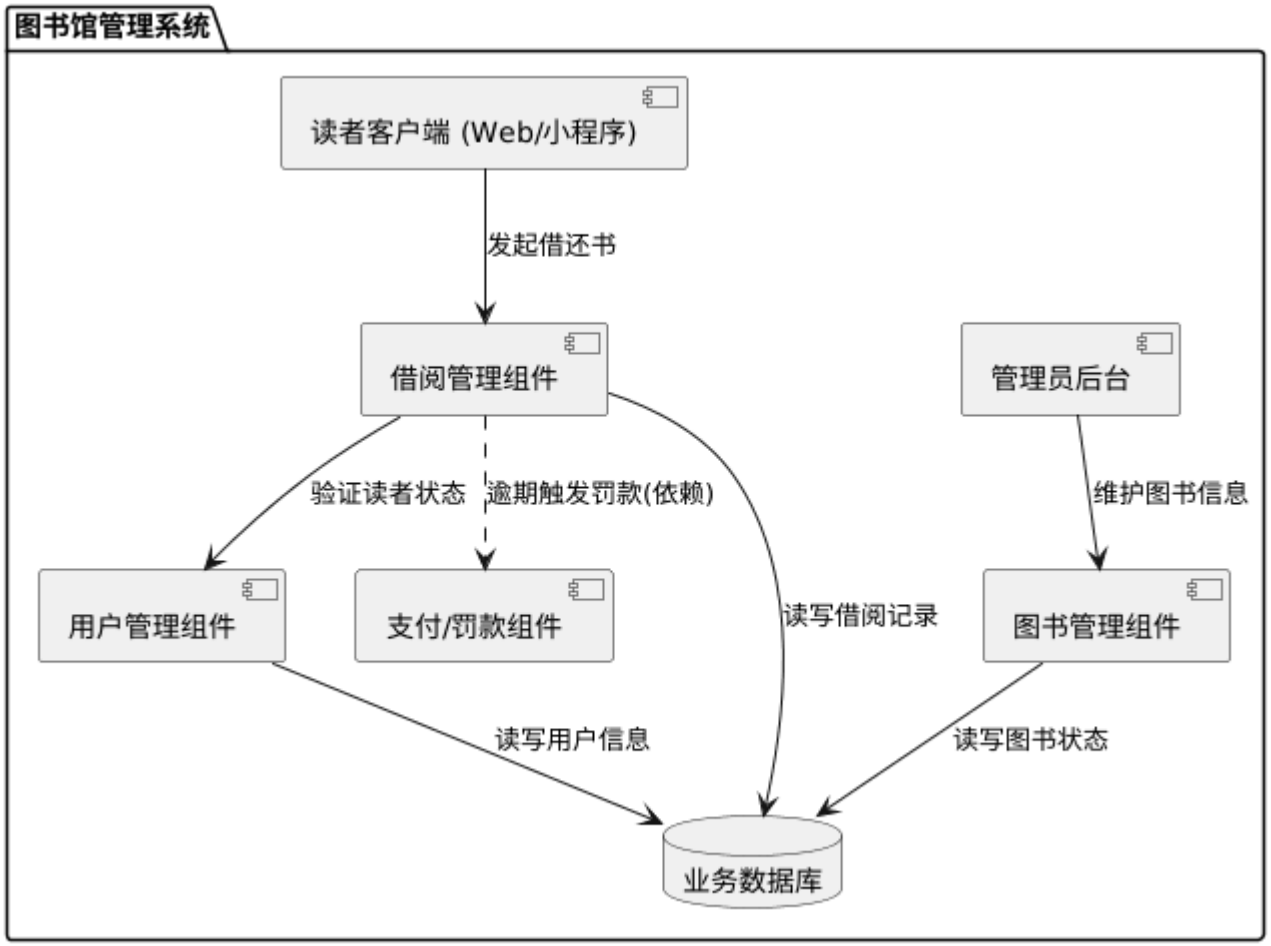
  database "业务数据库" as DB
}

ReaderApp --> BorrowComp : 发起借还书
AdminApp --> BookComp : 维护图书信息

BorrowComp --> UserComp : 验证读者状态
BorrowComp --> DB : 读写借阅记录
BookComp --> DB : 读写图书状态
UserComp --> DB : 读写用户信息

BorrowComp ..> PayComp : 逾期触发罚款(依赖)

@enduml
```



8、Deployment Diagram (部署图)

```
node "读者手机/电脑" as Client {
  artifact "微信小程序/浏览器"
}
```

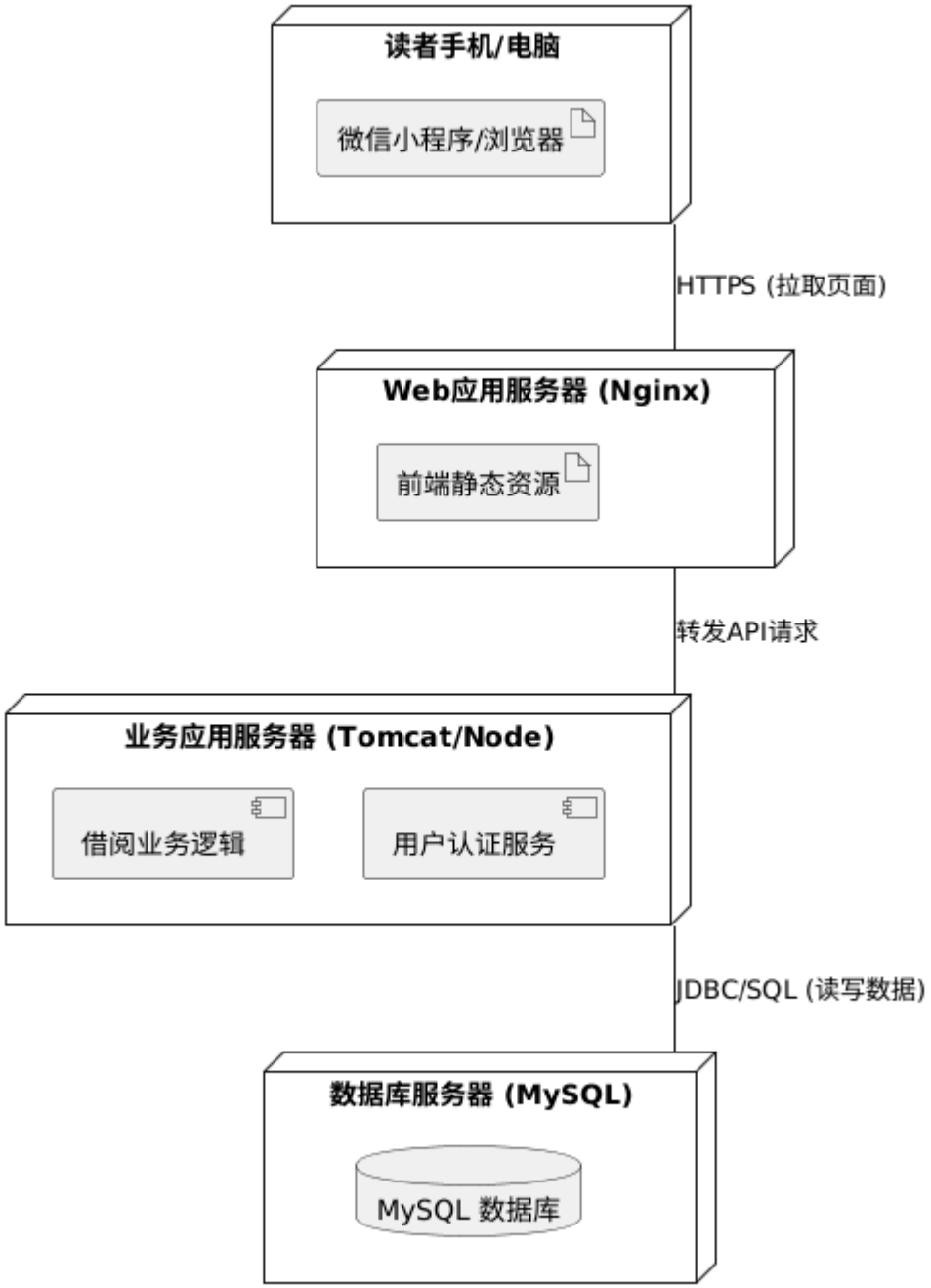
```
node "Web应用服务器 (Nginx)" as WebServer {
  artifact "前端静态资源"
}

node "业务应用服务器 (Tomcat/Node)" as AppServer {
  component "借阅业务逻辑"
  component "用户认证服务"
}

node "数据库服务器 (MySQL)" as DBServer {
  database "MySQL 数据库"
}

Client -- WebServer : HTTPS (拉取页面)
WebServer -- AppServer : 转发API请求
AppServer -- DBServer : JDBC/SQL (读写数据)

@enduml
```



9、State Diagram (状态图 / 状态机图)

```
[*] --> 在库 (可借)
在库 (可借) --> 已借出 : 读者借阅
已借出 --> 在库 (可借) : 正常归还
已借出 --> 逾期 : 超过截止日期未还
逾期 --> 在库 (可借) : 归还并缴纳罚款
逾期 --> 遗失/损坏 : 读者声明丢失
遗失/损坏 --> [*] : 管理员注销
@enduml
```

```
PlantUML version 1.2026.9beta4 / 87fe16c [2026-09-25 19:18:22 UTC]
[From string (line 2) ]

@startuml
[*] --> 在库 (可借)
 Syntax Error? (Assumed diagram type: sequence)
```

10、Timing Diagram (时间图 / 定时图)

```
robust "图书状态" as Book
robust "读者账号状态" as Reader

@0
Book is "在库"
Reader is "正常"

@10
Book is "已借出"
Reader is "借阅中"

@30
Book is "已借出"
Reader is "逾期违规"

@40
Book is "在库"
Reader is "正常(已还书)"

@enduml
```

